

Rockchip Developer Guide Android Car Multi-Display

文件标识: RK-YH-YF-751

发布版本: V1.0

日期: 2024-03-30

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

该文档主要是对RK3588显示规格信息和车载多屏显示方案架构进行简单介绍，并详细说明如何基于RK3588M车载SDK进行配置并实现多屏产品方案，同时附带多屏显示的常见问题解决方法说明，从而提高多屏产品开发的效率。

产品版本

芯片名称	内核版本
RK3588M	Kernel 5.10/Kernel 6.1
RK3576M	Kernel 6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0	罗伟	2024-03-30	初始发布

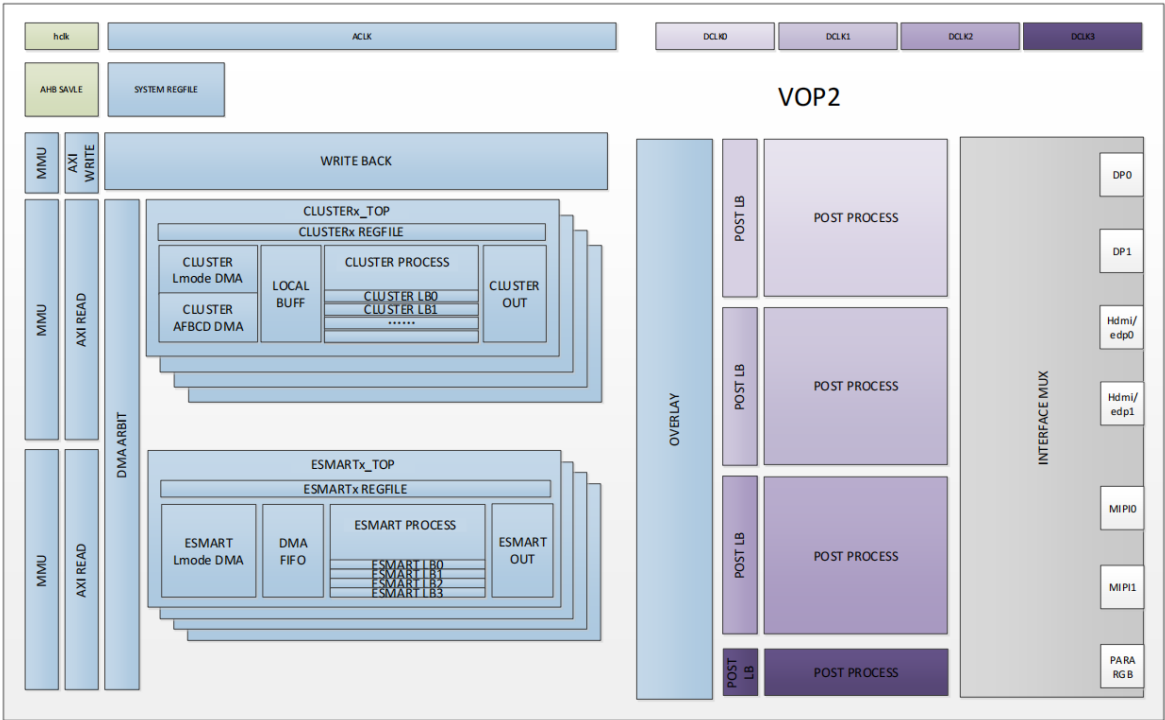
目录

- 1. 硬件规格说明
- 2. RK3588多屏异显软件方案介绍
 - 2.1 镜像模式与分割模式
 - 2.2 车载多屏显示驱动介绍
 - 2.2.1 车载显示框图介绍
 - 2.2.2 SERDES显示驱动介绍
 - 2.2.3 车载显示DTSI介绍
- 3. 多屏显示配置和实现方法说明
 - 3.1 配置说明
 - 3.1.1 内核配置说明
 - 3.1.2 XML配置说明
 - 3.1.2.1 硬件环境说明
 - 3.1.2.2 显示模式说明
 - 3.1.2.3 显示接口说明
 - 3.2 多屏拼接实现
 - 3.3 多屏异显实现
- 4. 常见问题和需求处理
 - 4.1 SERDES屏幕点不亮排查思路

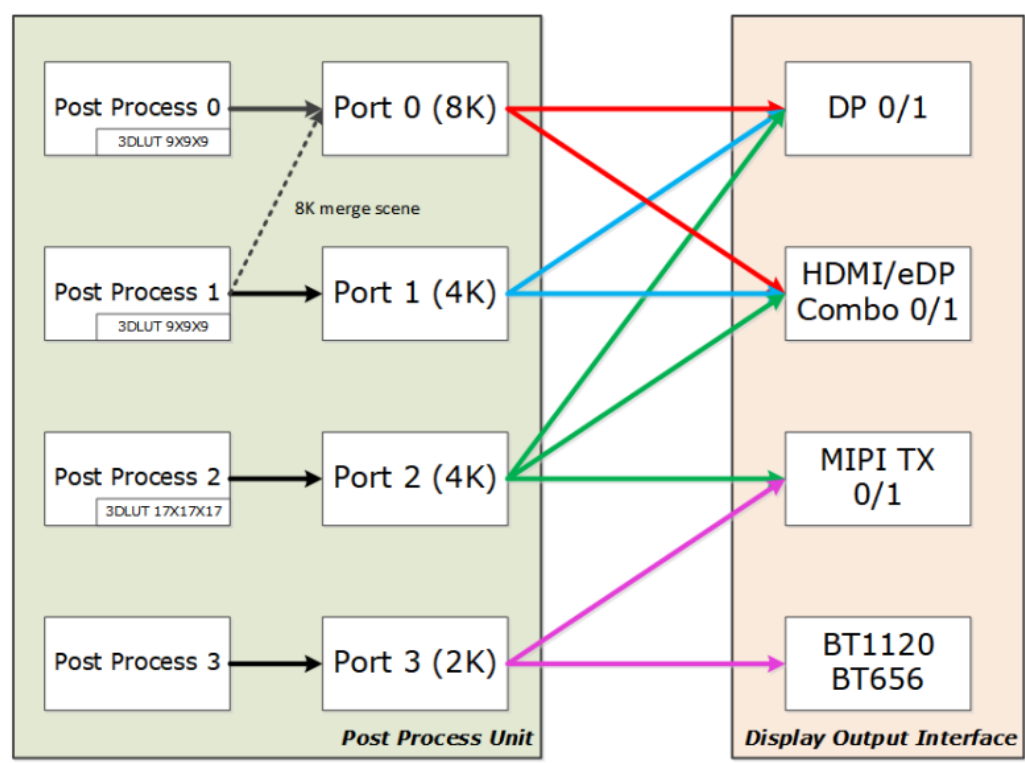
1. 硬件规格说明

RK3588支持8K显示输出，且接口丰富（包括DP0/1,EDP0/1, HDMI0/1, MIPI0/1, BT112/656），可以支持多屏异显或多屏拼接，考虑到内部VP的数量和引脚复用情况，RK3588最多可以支持7屏异显（例如7个1920x1080）或拼接。该文档主要是对RK3588显示规格信息和多屏软件方案架构进行简单介绍，并详细说明如何基于RK3588 SDK进行配置并实现多屏产品方案，同时附带多屏显示的常见问题解决方法说明，从而提高多屏产品开发的效率。

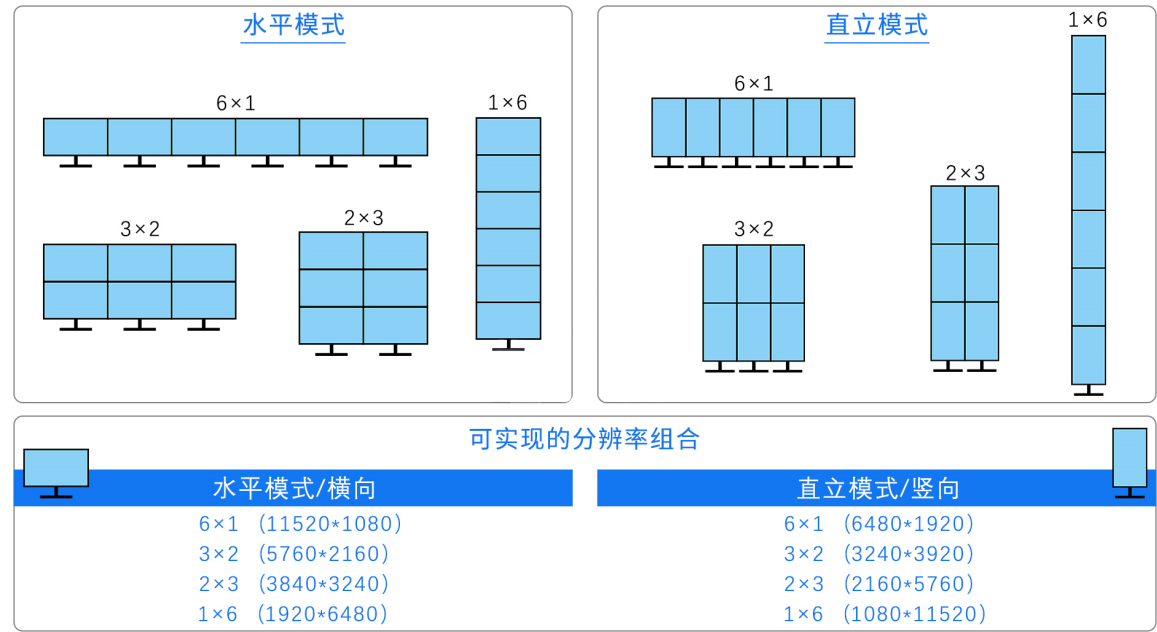
RK3588芯片内部VOP框图如下：



RK3588内部VP与显示接口的映射关系图（在DTS中给显示接口配置VP时需要注意这个映射关系）如下：



RK3588六屏显示的典型用法如下图：



RK3588六屏拼接（5760x2160）的效果图：



RK3588显示接口的主要规格如下:

接口类型	分辨率	协议标准
RGB	1920x1080@60	支持BT.656/BT.1120
MIPI	2560x1600@60/3840x2160@60	支持单/双通道，支持DSI v1.1, DCS v1.1, DPHY v2.0协议标准
eDP	3840x2160@60	支持DP1.2a和eDP1.3协议标准
HDMI	7680x4320@60	支持HDMI2.1协议标准
DP	7680x4320@30	支持DP1.4协议标准

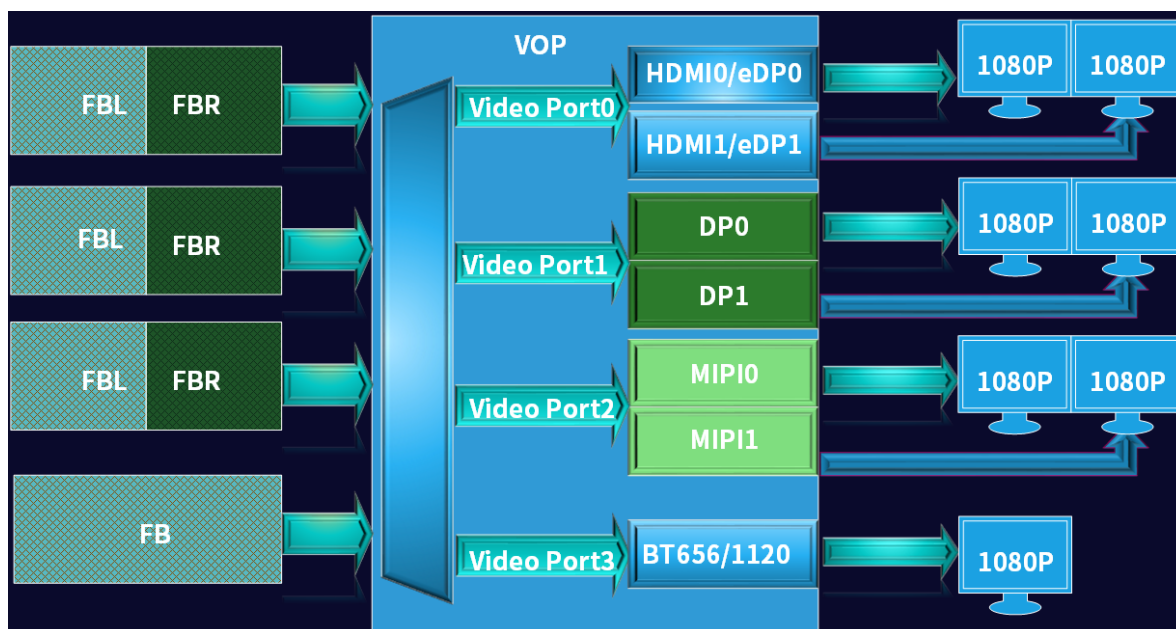
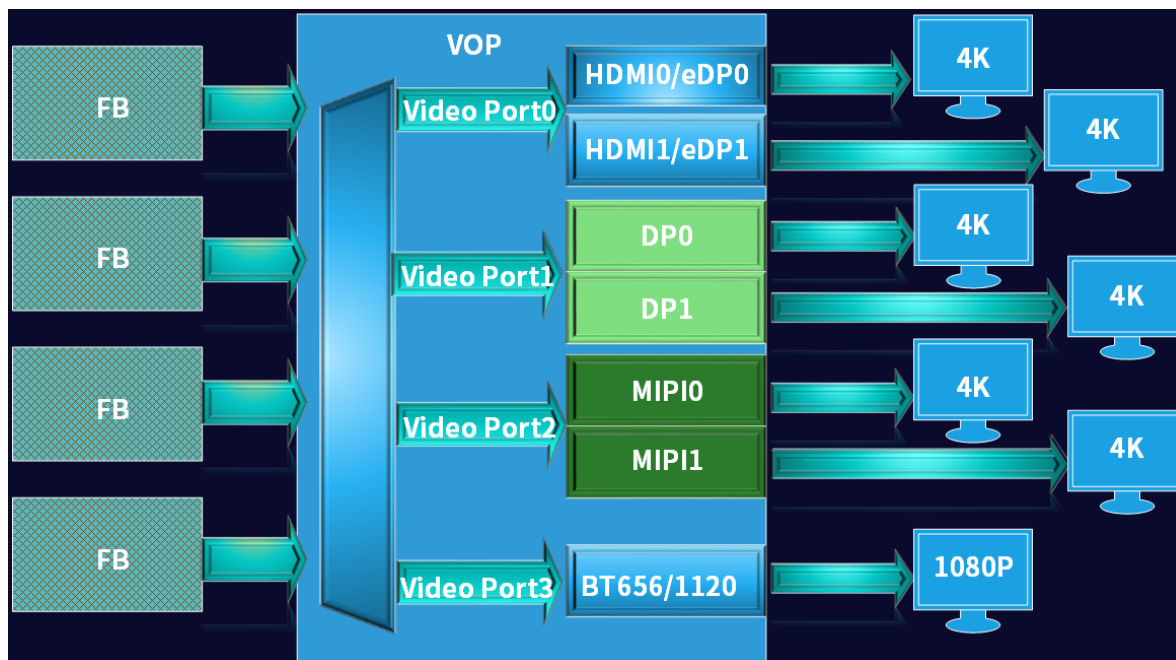
更多Rockchip芯片及更详细的显示规格信息请查阅文档
Rockchip_Developer_Guide_DRM_Display_Driver_CN.pdf。

2. RK3588多屏异显软件方案介绍

本章节旨在对多屏显示驱动、HWC层多屏处理流程、显示框架层多屏机制及多屏显示应用等进行简单而系统的介绍，从而让开发者对RK3588多屏显示软件架构有一个直观的理解，帮助提高车载多屏产品的开发效率。

2.1 镜像模式与分割模式

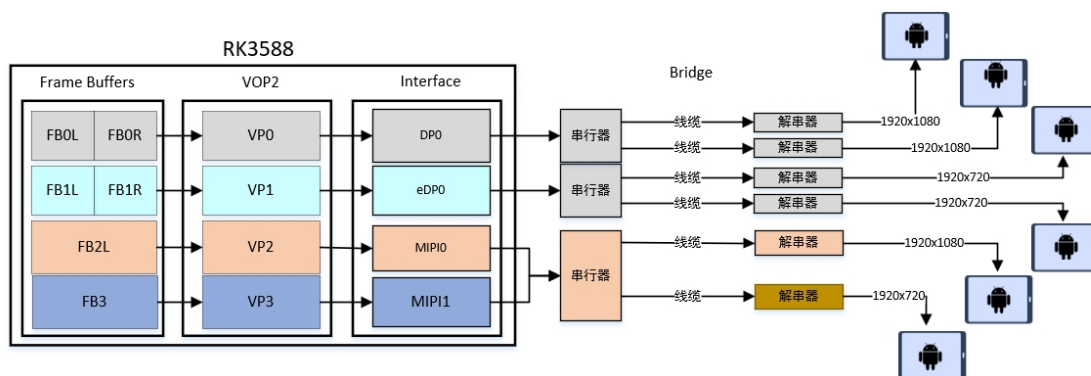
RK3588内部有4个VP，分别为VP0/VP1/VP2/VP3,其中VP0/VP1/VP2支持左右分割（不支持上下分割）模式，VP3不支持分割模式，所以如果不采用分割模式，RK3588可以支持4个异显，如果采用分割模式后最大可以支持7个屏幕的异显。以下两个图可以很好表达镜像模式与分割模式的区别。



2.2 车载多屏显示驱动介绍

2.2.1 车载显示框图介绍

由于在车上屏幕和主板之间距离较远，普通的显示方案无法满足要求，因此引入了串行器和解串器方案解决此问题，SERDES多屏显示方案不仅支持一进一出，也支持一进二出以及二进二出，以RK3588驱动多个SERDES屏幕为例，框图如下：



其中框图左半部分的介绍和使用方法参考RK3588的相关显示文档，右半部分为SERDES及显示，相关软件及使用方法则是本文档要介绍的重点。串行器将并行型号转成高频的串行信号，通过同轴电缆（最长可以达到15米）送给解串器，由解串器将串行信号还原成并行信号，并驱动和点亮最右侧的显示屏幕，并透传触摸信号。

2.2.2 SERDES显示驱动介绍

本文档介绍的SERDES软件架构为ROCKCHIP原创，实现了bridge, panel, pinctrl, gpio, irq, core几个通用模块，以及不同芯片型号的SERDES芯片接口，旨在使用一套软件框架兼容不同的Serdes型号，很大程度上减少每个客户的SERDES调试的重复劳动。其中bridge是串行器的驱动，panel是解串器的驱动，pinctrl是serdes芯片的iomux驱动，gpio是serdes芯片扩展GPIO的驱动，irq是serdes的中断处理机制，core是serdes驱动的核心接口，其余目录则包括了不同厂家的serdes芯片驱动，其接口用于被前面提到的不同模块调用，从而实现一套架构兼容不同的SERDES芯片。目前已经调好的包括ROHM和MAXIM的一系列芯片，其他的SERDES型号还在增加中。

驱动代码结构如下：

```
drivers/mfd/display-serdes/
```

驱动架构中最核心的部分为serdes结构体，了解本驱动架构前需要先了解其结构体，本驱动最核心的结构体简单介绍如下：

```
drivers/mfd/display-serdes/core.h
struct serdes {
    int num_gpio;
    struct mutex io_lock;
    struct mutex irq_lock;
    struct mutex wq_lock;
    struct mutex reg_check_lock;
    struct device *dev;
    enum serdes_type type;
    struct regmap *regmap;
    struct regulator *supply;
    struct extcon_dev *extcon;
    atomic_t conn_trigger;

    /* serdes power and reset pin */
    struct gpio_desc *reset_gpio; //电源控制相关
    struct gpio_desc *enable_gpio;
    struct regulator *vpower;
```



```

/* serdes irq pin */
struct gpio_desc *lock_gpio;    //中断相关变量
struct gpio_desc *err_gpio;
int lock_irq;
int err_irq;
int lock_irq_trig;
int err_irq_trig;
atomic_t flag_ser_init;

struct workqueue_struct *mfd_wq;    //中断工作队列
struct delayed_work mfd_delay_work;
bool route_enable;
bool use_delay_work;

struct kthread_worker *kworker;    //寄存器检测工作队列
struct kthread_delayed_work reg_check_work;
bool use_reg_check_work;

bool split_mode_enable;    //一拖二模式相关控制变量
unsigned int reg_hw;
unsigned int reg_use;
unsigned int link_use;
unsigned int id_serdes_bridge_split;
unsigned int id_serdes_panel_split;
struct serdes *g_serdes_bridge_split;

struct pinctrl *pinctrl_node;    //pinctrl驱动相关
struct pinctrl_state *pins_default;
struct pinctrl_state *pins_init;
struct pinctrl_state *pins_sleep;

struct serdes_init_seq *serdes_init_seq;    //初始化序列
struct serdes_init_seq *serdes_backup_seq;    //初始化序列备份
struct serdes_bridge *serdes_bridge;    //一拖一的bridge结构体, 对应串行器A
struct serdes_bridge_split *serdes_bridge_split;    //一拖二的bridge结构体, 对应
串行器B
struct serdes_panel *serdes_panel;    //一拖一的panel结构体, 对应解串器A
struct serdes_panel_split *serdes_panel_split;    //一拖二下的panel结构体, 对应解串
器B
struct serdes_pinctrl *pinctrl;    //pinctrl结构体, 用于可视化配置SERDES寄存器
struct serdes_chip_data *chip_data;    //对应不同serdes芯片, 用于为bridge/panel等模
块提供接口
};

```

2.2.3 车载显示DTSI介绍

DTSI与驱动结构是对应关系, 即将串行器和解串器作为独立单元进行配置, 配置内容包括初始化序列、pinctrl、gpio、bridge、panel等几个部分, 下面以max96789/max96752为例说明显示dtsi结构, 其他的SERDES的DTSI结构也是基本相同的。

```

&i2c8 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c8m2_xfer>;

```



```

clock-frequency = <400000>;

i2c8_max96789: i2c8-max96789@42 {
    compatible = "maxim,max96789";
    reg = <0x42>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c8_serdes_pins>;
    lock-gpios = <&gpio2 RK_PC7 GPIO_ACTIVE_HIGH>;
    sel-mipi;
    id-serdes-bridge-split = <0x01>;
    status = "okay";

    serdes-init-sequence = [
        //串行器MAX96789的初始化序列
        //Disable Video pipe
        0002 0003
        . . . . .
        //Turn on video pipe
        0002 0033
        //Enable splitter mode reset one shot
        0010 0023
        ffff f000 //0xf000 ms delay
    ];

    i2c8_max96789_pinctrl: i2c8-max96789-pinctrl {
        compatible = "maxim,max96789-pinctrl";
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&i2c8_max96789_pinctrl_pins>;
        pinctrl-1 = <&i2c8_max96789_pinctrl_pins>;
        status = "okay";
        i2c8_max96789_pinctrl_pins: pinctrl-pins {
            . . . . .
            tp-int {
                pins = "MAX96789_MFP8"; //SERDES的可视化配置
                function = "DES_RXID8_TO_SER"; //ID=8,和MAX96752的GPIO2透传
            };
            . . . . .
        };

        i2c8_max96789_gpio: i2c8-max96789-gpio {
            compatible = "maxim,max96789-gpio";
            status = "okay";

            gpio-controller;
            #gpio-cells = <2>;
            gpio-ranges = <&i2c8_max96789_pinctrl 0 160 20>; //GPIO定义不能
和其他重复
        };
    };

    i2c8_max96789_bridge: i2c8-max96789-bridge {
        compatible = "maxim,max96789-bridge";
        status = "okay";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

```

```

        port@0 {
            reg = <0>;
            i2c8_max96789_from_dsi: endpoint {
                remote-endpoint = <&dsi_out_i2c8_max96789>;
            };
        };

        port@1 {
            reg = <1>;
            i2c8_max96789_out_i2c8_max96752: endpoint {
                remote-endpoint = <&i2c8_max96752_from_i2c8_max96789>;
            };
        };
    };

};

i2c8_max96752: i2c8-max96752@4a {
    compatible = "maxim,max96752";
    reg = <0x4a>;
    //reg-hw = <0x4a>;
    id-serdes-panel-split = <0x01>;
    link = <0x01>;
    status = "okay";

    serdes-init-sequence = [
        /*解串器max96752初始化序列*/
        . . . . .
    ];

    i2c8_max96752_pinctrl: i2c8-max96752-pinctrl {
        compatible = "maxim,max96752-pinctrl";
        pinctrl-names = "default", "init", "sleep";
        pinctrl-0 = <&i2c8_max96752_panel_pins>;
        pinctrl-1 = <&i2c8_max96752_panel_pins>;
        pinctrl-2 = <&i2c8_max96752_panel_sleep_pins>;
        status = "okay";

        i2c8_max96752_panel_pins: panel-pins {
            . . . . .
            tp-int {
                pins = "MAX96752_GPIO2";
                function = "DES_TXID8_TO_SER"; //ID=8,和MAX96789的MFP8透传
            };
            . . . . .
        };

        i2c8_max96752_panel_sleep_pins: panel-sleep-pins {
            . . . . .
        };

        i2c8_max96752_gpio: i2c8-max96752-gpio {
            compatible = "maxim,max96752-gpio";
            status = "okay";

            gpio-controller;
            #gpio-cells = <2>;

```

```

        gpio-ranges = <&i2c8_max96752_pinctrl 0 180 15>; //GPIO定义不能
和其他重复

    };

};

i2c8_max96752_panel: i2c8-max96752-panel {
    compatible = "maxim,max96752-panel";
    status = "okay";

    backlight = <&backlight>;
    panel-size = <346 194>;
    //屏幕时序
    panel-timing {
        clock-frequency = <115000000>;
        hactive = <1920>;
        vactive = <720>;
        hfront-porch = <56>;
        hsync-len = <32>;
        hback-porch = <56>;
        vfront-porch = <200>;
        vsync-len = <2>;
        vback-porch = <8>;
        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <0>;
        pixelclk-active = <0>;
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            i2c8_max96752_from_i2c8_max96789: endpoint {
                remote-endpoint = <&i2c8_max96789_out_i2c8_max96752>;
            };
        };
    };
};

//触摸DTS配置，其I2C地址可以被SERDES芯片重定义
himax@45 {
    compatible = "himax,hxcommon";
    reg = <0x45>;
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&touch_gpio_dsi>;
    pinctrl-1 = <&touch_gpio_dsi>;
    himax,location = "himax-touch-dsi";
    himax,irq-gpio = <&gpio3 RK_PA3 IRQ_TYPE_EDGE_FALLING>;
    //该GPIO配置会调用到SERDES扩展GPIO驱动
    himax,rst-gpio = <&i2c8_max96752_gpio 5 GPIO_ACTIVE_LOW>;
    himax,panel-coords = <0 1920 0 720>;
    himax,display-coords = <0 1920 0 720>;
    status = "okay";
};
};

```

3. 多屏显示配置和实现方法说明

RK3588支持多屏显示，具体应用上可以分为多屏同显、多屏异显、多屏拼接，其中多屏拼接可以看成是多屏异显的一个特殊情况。对于产品端实现此多屏产品的思路是首先把6个屏幕都调试好并能正常显示，这其中可能会涉及到硬件显示接口转换的驱动问题，也可能需要根据需要配置VP和SPLIT模式。其次，根据需求修改HAL层源文件配置hardware/rockchip/hwcomposer/drmhwc2/HwComposerEnv.xml，该XML文件主要用于描述显示模式（同显/异显/拼接）、宽和高、显示接口数量及名称、每个显示的位置等信息，后面部分会对这部分再详细说明。

3.1 配置说明

3.1.1 内核配置说明

DTS显示相关的配置主要是显示的开关、显示接口与VP的映射关系的设置、是否采用分割模式等。例如下面的代码段是对edp0和edp1进行配置，edp0和edp1共用vp1且采用分割模式，对于其他模块依次类推处理同时做好VP的分配即可。

```
&edp0 {
    split-mode;
    force-hpd;
    status = "okay";
};

&edp0_in_vp0 {
    status = "disabled";
};

&edp0_in_vp1 {
    status = "okay";
};

&edp0_in_vp2 {
    status = "disabled";
};

&edp1 {
    force-hpd;
    status = "okay";
};

&edp1_in_vp0 {
    status = "disabled";
};

&edp1_in_vp1 {
    status = "okay";
};

&edp1_in_vp2 {
```

```
        status = "disabled";
    };
};
```

下面的例子是将RK3588-EVB1支持3*2的六屏拼接的关键DTS配置修改，其中HDMI0/HDMI1采用分割模式共用VP0，DP0/DP1采用分割模式共用VP1，DSI0采用VP2，DSI1采用VP3，VP可以根据实际需求进行分配。

```
--- a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
@@ -247,13 +247,15 @@
    };

    &dp0 {
+       split-mode;
        status = "okay";
    };

-    &dp0_in_vp2 {
+    &dp0_in_vp1 {
        status = "okay";
    };

    &dp0_sound{
        status = "okay";
    };
@@ -265,7 +267,7 @@
        status = "okay";
    };

-    &dp1_in_vp2 {
+    &dp1_in_vp1 {
        status = "okay";
    };

@@ -278,11 +280,11 @@
    };

    &dsi0_in_vp2 {
-       status = "disabled";
+       status = "okay";
    };

    &dsi0_in_vp3 {
-       status = "okay";
+       status = "disabled";
    };

    &dsi0_panel {
@@ -297,7 +299,7 @@
        * when dsi1 is enabled
        */
        &dsi1 {
-       status = "disabled";
+       status = "okay";
        };

        &dsi1_in_vp2 {
```

```

@@ -305,7 +307,7 @@
};

&dsil_in_vp3 {
-     status = "disabled";
+     status = "okay";
};

&dsil_panel {
@@ -347,6 +349,7 @@
};

&hdmi0 {
+     split-mode;
     enable-gpios = <&gpio4 RK_PB1 GPIO_ACTIVE_HIGH>;
     status = "okay";
};
@@ -364,7 +367,7 @@
     status = "okay";
};

-&hdmi1_in_vp1 {
+&hdmi1_in_vp0 {
     status = "okay";
};

```

3.1.2 XML配置说明

考虑到RK3588的显示接口丰富，需求可变性较大，如果依靠以往的属性方式，比较难以描述，因此采用了XML的描述方式。配置文件位于：`hardware/rockchip/hwcomposer/drmhwc2/HwComposerEnv.xml`，完整编译后会安装到设备 `/vendor/etc/HwComposerEnv.xml` 位置，但如果 `device/rockchip/rk3588/rk3588m_u/displaysHwComposerEnv-multidisplay.xml` 文件存在，则以这个文件为优先。

3.1.2.1 硬件环境说明

终端设备为6台1080p HDMI电视，芯片内部注册为4个Connector设备，对应的ConnectorType-TypeId与分辨率如下：

ConnectorType-TypeId	设备分辨率	Vop-SpiltMode	备注
HDMI-A-1	3840x1080p60	开启	Vop-SpiltMode将分割为2路1920x1080p60信号
HDMI-A-2	1920x1080p60	关闭	无
DP-1	3840x1080p60	开启	Vop-SpiltMode将分割为2路1920x1080p60信号
HDMI-A-3	1920x1080p60	关闭	

3.1.2.2 显示模式说明

```
<DsisplayMode Mode="1" FbWidth="5760" FbHeight="2160" ConnectorCnt="4">
```

当前Mode 设置为1=Slicing，表示为拼接模式，其中0/1/2 依次表示为同显，拼接和异显三个模式。

FbWidth / FbHeight，系统渲染分辨率，完整的图像尺寸，具体数值通过以下方式计算得来：

```
FbWidth = 1920 * 3 = 5760    // 3x2 布局的拼接屏幕
FbHeight= 1080 * 2 = 2160    // 3x2 布局的拼接屏幕
ConnectorCnt = 4              // 为系统内部注册的Connector数量
```

3.1.2.3 显示接口说明

```
<Connector>
  <Type>HDMI-A</Type>
  <TypeId>1</TypeId>
  <SrcX>0</SrcX>    <!-- Framebuffer x Offset -->
  <SrcY>0</SrcY>    <!-- Framebuffer y Offset -->
  <SrcW>3840</SrcW> <!-- Framebuffer Width -->
  <SrcH>1080</SrcH> <!-- Framebuffer Height-->
  <DstX>0</DstX>    <!-- unuse , reserved -->
  <DstY>0</DstY>    <!-- unuse , reserved -->
  <DstW>0</DstW>    <!-- unuse , reserved -->
  <DstH>0</DstH>    <!-- unuse , reserved -->
</Connector>
```

1. ConnectorType 与 TypeId 信息配置

Connector配置需要的所有信息均可通过 modetest 工具获得，modetest的具体信息如下：

```
# modetest 编译
mmm external/libdrm/tests/modetest

# modetest 输出目录
$OUT/data/nativetest64/modetest/modetest # 64位
$OUT/data/nativetest/modetest/modetest   # 32位

# 获取Drm Driver注册的所有Connector信息
adb shell modetest -c > modetest-connector.log

# 具体日志如下：
-----
Connectors:
id      encoder status      name      size (mm)      modes      encoders
409     408      connected    HDMI-A-1    700x390      26         408
  modes:
  index name      refresh (Hz) hdisp hss  hse  htot vdisp vss  vse  vtot
  #0    3840x1080 60.00 3840 4016 4104 4400 1080 1084 1089 1125 297000
flags: ...
(...)

419     418      connected    HDMI-A-2    510x290      10         418
  modes:
  index name      refresh (Hz) hdisp hss  hse  htot vdisp vss  vse  vtot
```



```
#0      1920x1080 60.00      1920  2008 2052 2200 1080  1084 1089 1125
148500 flags: ...
(...)

421      420      connected      HDMI-A-3      510x290      10      420
modes:
index name      refresh (Hz) hdisp hss  hse  htot vdisp vss  vse  vtot
#0      1920x1080 60.00      1920  2008 2052 2200 1080  1084 1089 1125
148500 flags: ...
(...)

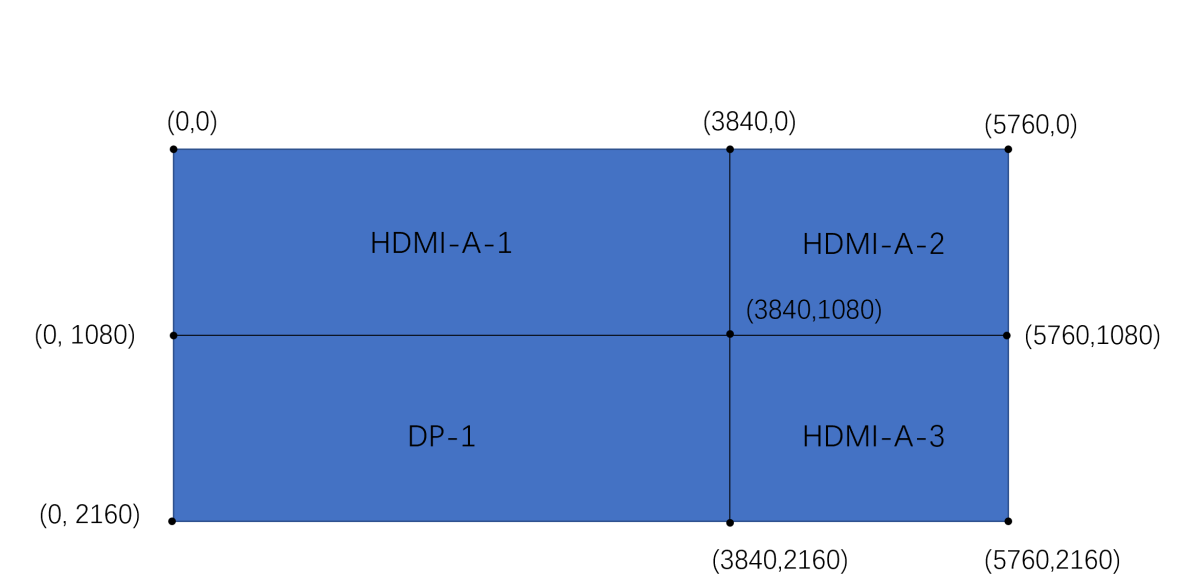
423      422      connected      DP-1      1020x290      12      422
modes:
index name      refresh (Hz) hdisp hss  hse  htot vdisp vss  vse  vtot
#1      3840x1080 60.00      3840  4016 4104 4400 1080  1084 1089 1125 297000
flags: ..

-----

# ConnectorType 与 TypeId 从 Connector name 描述中获取: HDMI-A-1 / HDMI-A-2 /
HDMI-A-3 / DP-1
# 对应关系如下:
#      HDMI-A-1: ConnectorType = HDMI-A , TypeId = 1
#      HDMI-A-2: ConnectorType = HDMI-A , TypeId = 2
#      HDMI-A-3: ConnectorType = HDMI-A , TypeId = 3
#      DP-1    : ConnectorType = DP      , TypeId = 1
```

2. SrcX/Y/W/H 与 DstX/Y/W/H 图形参数配置

系统渲染的分辨率设置为 5760x2160，那么对应Connector 显示的图像区域如下图：



根据上图，就能够很容易的得出4个 Connector 的 Src Info坐标：

ConnectorType-TypeId	SrcX	SrcY	SrcW	SrcH
HDMI-A-1	0	0	3840	1080
HDMI-A-2	3840	0	1920	1080
DP-1	0	1080	3840	1080
HDMI-A-3	3840	1080	1920	1080

3.2 多屏拼接实现

RK3588最高可以支持到7屏拼接（如果还不够，客户可以自行采用第三方硬件设备扩展），下面的图片是当前六屏拼接的现状，其中绿色部分已支持，红色部分理论支持待实现。



下面仍然以3x2 模式（5760x2160）为例说明，下图为3x2模式的cat /d/dri/0/summary 信息（使用多个显示接口的6个1920x1080），该信息会用于XML配置，其表示了HDMI0/HDMI1各输出1920x1080，DP0/DP1各输出1920x1080，DSI2HDMI1输出1920x1080，DSI2HDMI2输出1920x1080。

```

Video Port0: ACTIVE
Connector: HDMI-A-1
bus_format[100a]: RGB888_1x24
overlay_mode[0] output_mode[f] color_space[0], eotf:0
Display mode: 3840x1080p60
clk[297000] real_clk[297000] type[8] flag[5]
H: 3840 4016 4104 4400
V: 1080 1084 1089 1125
Cluster0-win0: ACTIVE
win_id: 0
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0] glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[0, 0] rect[3840 x 1080]
dst: pos[0, 0] rect[3840 x 1080]
buf[0]: addr: 0x00000000070f2000 pitch: 46080 offset: 0
Video Port1: ACTIVE
Connector: DP-1
bus_format[100a]: RGB888_1x24
overlay_mode[0] output_mode[f] color_space[0], eotf:0
Display mode: 3840x1080p60
clk[297000] real_clk[297000] type[40] flag[5]
H: 3840 4016 4104 4400
V: 1080 1084 1089 1125
Cluster3-win0: ACTIVE
win_id: 6
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0] glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[7680, 0] rect[3840 x 1080]
dst: pos[0, 0] rect[3840 x 1080]
buf[0]: addr: 0x0000000013332000 pitch: 46080 offset: 0
Video Port2: ACTIVE
Connector: HDMI-A-2
bus_format[100a]: RGB888_1x24
overlay_mode[0] output_mode[0] color_space[0], eotf:0
Display mode: 1920x1080p60
clk[148500] real_clk[148500] type[40] flag[5]
H: 1920 2008 2052 2200
V: 1080 1084 1089 1125
Cluster1-win0: ACTIVE
win_id: 2
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0] glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[3840, 0] rect[1920 x 1080]
dst: pos[0, 0] rect[1920 x 1080]
buf[0]: addr: 0x000000000d212000 pitch: 46080 offset: 0
Video Port3: ACTIVE
Connector: HDMI-A-3
bus_format[100a]: RGB888_1x24
overlay_mode[0] output_mode[0] color_space[0], eotf:0
Display mode: 1920x1080p60
clk[148500] real_clk[148500] type[40] flag[5]
H: 1920 2008 2052 2200
V: 1080 1084 1089 1125
Cluster2-win0: ACTIVE
win_id: 4
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0] glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[5760, 0] rect[1920 x 1080]

```

对应的3x2配置文件HwComposerEnv.xml范例如下：

```

<?xml version="1.0" encoding="utf-8"?>
<!-- HwComposerEnv module xml -->
<HwComposerEnv Version="1.1.1" >
  <!--
    DsisplayMode:
      Mode: 0=None 1=Slicing 2=Presentation
      FbWidth: Framebuffer Width
      FbHeight: Framebuffer Width
      ConnectorCnt: display count
  -->
  <DsisplayMode Mode="1" FbWidth="5760" FbHeight="2160" ConnectorCnt="4">
    <Connector>
      <Type>HDMI-A</Type>
      <TypeId>1</TypeId>
      <SrcX>0</SrcX>      <!-- Framebuffer x Offset -->
      <SrcY>0</SrcY>      <!-- Framebuffer y Offset -->
      <SrcW>3840</SrcW>    <!-- Framebuffer Width -->
      <SrcH>1080</SrcH>    <!-- Framebuffer Height-->
    </Connector>
  </DsisplayMode>
</HwComposerEnv>

```

```

        <DstX>0</DstX>      <!-- unuse , reserved -->
        <DstY>0</DstY>      <!-- unuse , reserved -->
        <DstW>0</DstW>      <!-- unuse , reserved -->
        <DstH>0</DstH>      <!-- unuse , reserved -->
    </Connector>
    <Connector>
        <Type>HDMI-A</Type>
        <TypeId>2</TypeId>
        <SrcX>3840</SrcX> <!-- Framebuffer x Offset -->
        <SrcY>0</SrcY>    <!-- Framebuffer y Offset -->
        <SrcW>1920</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->
        <DstX>0</DstX>    <!-- unuse , reserved-->
        <DstY>0</DstY>    <!-- unuse , reserved-->
        <DstW>0</DstW>    <!-- unuse , reserved-->
        <DstH>0</DstH>    <!-- unuse , reserved-->
    </Connector>
    <Connector>
        <Type>DP</Type>
        <TypeId>1</TypeId>
        <SrcX>0</SrcX>    <!-- Framebuffer x Offset -->
        <SrcY>1080</SrcY> <!-- Framebuffer y Offset -->
        <SrcW>3840</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->
        <DstX>0</DstX>    <!-- unuse , reserved-->
        <DstY>0</DstY>    <!-- unuse , reserved-->
        <DstW>0</DstW>    <!-- unuse , reserved-->
        <DstH>0</DstH>    <!-- unuse , reserved-->
    </Connector>
    <Connector>
        <Type>HDMI-A</Type>
        <TypeId>3</TypeId>
        <SrcX>3840</SrcX> <!-- Framebuffer x Offset -->
        <SrcY>1080</SrcY> <!-- Framebuffer y Offset -->
        <SrcW>1920</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->
        <DstX>0</DstX>    <!-- unuse , reserved-->
        <DstY>0</DstY>    <!-- unuse , reserved-->
        <DstW>0</DstW>    <!-- unuse , reserved-->
        <DstH>0</DstH>    <!-- unuse , reserved-->
    </Connector>
</DisplayMode>
</HwComposerEnv>

```

假如需要支持6x1模式（11520x1080），XML需要进行修改，修改如下图：

```

--- a/display-layout/HwComposerEnv.xml
+++ b/display-layout/HwComposerEnv.xml
@@ -31,7 +31,7 @@
|      3840      |      1920      |      - FbWidth = 5760
|                  |                  |      - ConnectorCnt = 4
-->
- <DisplayMode Mode="1" FbWidth="5760" FbHeight="2160" ConnectorCnt="4">
+ <DisplayMode Mode="1" FbWidth="11520" FbHeight="1080" ConnectorCnt="4">
    <Connector>
        <!--
            Example 1 - Slicing-6x1 1920x1080:

```

```

@@ -88,8 +88,8 @@
    <Connector>
        <Type>DP</Type>
        <TypeId>1</TypeId>
-       <SrcX>0</SrcX> <!-- Framebuffer x Offset -->
-       <SrcY>1080</SrcY> <!-- Framebuffer y Offset -->
+       <SrcX>3840</SrcX> <!-- Framebuffer x Offset -->
+       <SrcY>0</SrcY> <!-- Framebuffer y Offset -->
        <SrcW>3840</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->
        <DstX>0</DstX> <!-- unused , reserved-->
@@ -100,7 +100,7 @@
    <Connector>
        <Type>HDMI-A</Type>
        <TypeId>2</TypeId>
-       <SrcX>3840</SrcX> <!-- Framebuffer x Offset -->
+       <SrcX>7680</SrcX> <!-- Framebuffer x Offset -->
        <SrcY>0</SrcY> <!-- Framebuffer y Offset -->
        <SrcW>1920</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->
@@ -112,8 +112,8 @@
    <Connector>
        <Type>HDMI-A</Type>
        <TypeId>3</TypeId>
-       <SrcX>3840</SrcX> <!-- Framebuffer x Offset -->
-       <SrcY>1080</SrcY> <!-- Framebuffer y Offset -->
+       <SrcX>9600</SrcX> <!-- Framebuffer x Offset -->
+       <SrcY>0</SrcY> <!-- Framebuffer y Offset -->
        <SrcW>1920</SrcW> <!-- Framebuffer Width -->
        <SrcH>1080</SrcH> <!-- Framebuffer Height-->

```

如果实现其他的拼接，方法类似，不一一作解释。

3.3 多屏异显实现

多屏拼接属于多屏异显的一个类型，而且不同屏幕显示的内容之间可能需要保持同步，而多屏异显则没有这个要求，一个屏幕对应到一个DisplayID，屏幕内容可以没有关联。对于底层部分的设置，和多屏拼接可以共用，但在XML配置上有区别。以下打印信息是6个屏幕的summary内容，其表示了EDP0/EDP1各输出1920x1080,DP0/DP1各输出1920x1080,DSI0输出1920x720,DSI1输出1920x384:

```

console:/ # cat /d/dri/0/summary
Video Port0: ACTIVE
Connector: DP-1
    bus_format[100a]: RGB888_1X24
    overlay_mode[0] output_mode[f] color_space[0], eotf:0
Display mode: 3840x1080p60
    clk[297000] real_clk[297000] type[48] flag[a]
    H: 3840 4120 4200 4400
    V: 1080 1095 1115 1125
Esmart0-win0: ACTIVE
    win_id: 8
    format: AB24 little-endian (0x34324241) SDR[0] color_space[0]
glb_alpha[0xff]
    rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
    csc: y2r[0] r2y[0] csc mode[0]

```

```
zpos: 1
src: pos[0, 0] rect[1920 x 1080]
dst: pos[1920, 0] rect[1920 x 1080]
buf[0]: addr: 0x00000000f11fd000 pitch: 7680 offset: 0
Cluster0-win0: ACTIVE
win_id: 0
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0]
glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[0, 0] rect[1920 x 1080]
dst: pos[0, 0] rect[1920 x 1080]
buf[0]: addr: 0x00000000f4cd8000 pitch: 7680 offset: 0
Video Port1: ACTIVE
Connector: eDP-1
bus_format[100a]: RGB888_1X24
overlay_mode[0] output_mode[f] color_space[0], eotf:0
Display mode: 3840x1080p60
clk[297000] real_clk[297000] type[48] flag[a]
H: 3840 4120 4200 4400
V: 1080 1095 1115 1125
Esmart1-win0: ACTIVE
win_id: 10
format: AB24 little-endian (0x34324241) SDR[0] color_space[0]
glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 1
src: pos[0, 0] rect[1920 x 1080]
dst: pos[1920, 0] rect[1920 x 1080]
buf[0]: addr: 0x00000000f0a14000 pitch: 7680 offset: 0
Cluster1-win0: ACTIVE
win_id: 2
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0]
glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[0, 0] rect[1920 x 1080]
dst: pos[0, 0] rect[1920 x 1080]
buf[0]: addr: 0x00000000f57cc000 pitch: 7680 offset: 0
Video Port2: ACTIVE
Connector: DSI-1
bus_format[100a]: RGB888_1X24
overlay_mode[0] output_mode[0] color_space[0], eotf:0
Display mode: 1920x720p59
clk[87000] real_clk[87000] type[48] flag[a]
H: 1920 1952 1962 1984
V: 720 730 734 741
Cluster2-win0: ACTIVE
win_id: 4
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0]
glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[0, 0] rect[1920 x 720]
```

```

        dst: pos[0, 0] rect[1920 x 720]
        buf[0]: addr: 0x00000000f477c000 pitch: 7680 offset: 0
Video Port3: ACTIVE
    Connector: DSI-2
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[0] color_space[0], eotf:0
Display mode: 1920x384p59
    clk[50390] real_clk[50000] type[48] flag[a]
    H: 1920 1946 1956 1972
    V: 384 404 407 431
Esmart3-win0: ACTIVE
    win_id: 11
    format: AB24 little-endian (0x34324241) SDR[0] color_space[0]
glb_alpha[0xff]
    rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
    csc: y2r[0] r2y[0] csc mode[0]
    zpos: 1
    src: pos[0, 0] rect[1920 x 384]
    dst: pos[0, 0] rect[1920 x 384]
    buf[0]: addr: 0x00000000f54f0000 pitch: 7680 offset: 0
Cluster3-win0: ACTIVE
    win_id: 6
    format: XB24 little-endian (0x34324258) [AFBC] SDR[0] color_space[0]
glb_alpha[0xff]
    rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
    csc: y2r[0] r2y[0] csc mode[0]
    zpos: 0
    src: pos[58, 422] rect[1164 x 436]
    dst: pos[448, 0] rect[1024 x 384]
    buf[0]: addr: 0x00000000f8283000 pitch: 5120 offset: 0

```

在XML配置方面，多屏异显配置相对简单点，主要有两个区别，第一，只需要对需要异显的接口进行配置，不需要的不用配置，例如对于本需求只需要对DP0/DP1和EDP0/EDP1进行异显即可（请注意将DP0/DP1和EDP0/EDP1设为split-mode）；第二就是Mode值不同，对于异显Mode值需要配置为2。本方案的6屏异显配置范例如下：

```

<?xml version="1.0" encoding="utf-8"?>
<!-- HwComposerEnv module xml -->
<HwComposerEnv Version="1.1.1" >
    <DsisplayMode Mode="2" FbWidth="0" FbHeight="0" ConnectorCnt="2">
        <Connector>
            <Type>DP</Type>
            <TypeId>1</TypeId>
            <SrcX>0</SrcX>    <!-- Framebuffer x Offset -->
            <SrcY>0</SrcY>    <!-- Framebuffer y Offset -->
            <SrcW>0</SrcW>    <!-- Framebuffer Width -->
            <SrcH>0</SrcH>    <!-- Framebuffer Height-->
            <DstX>0</DstX>    <!-- unuse , reserved -->
            <DstY>0</DstY>    <!-- unuse , reserved -->
            <DstW>0</DstW>    <!-- unuse , reserved -->
            <DstH>0</DstH>    <!-- unuse , reserved -->
        </Connector>
        <Connector>
            <Type>eDP</Type>
            <TypeId>1</TypeId>
            <SrcX>0</SrcX>    <!-- Framebuffer x Offset -->
            <SrcY>0</SrcY>    <!-- Framebuffer y Offset -->

```

```
<SrcW>0</SrcW> <!-- Framebuffer Width -->
<SrcH>0</SrcH> <!-- Framebuffer Height-->
<DstX>0</DstX> <!-- unuse , reserved-->
<DstY>0</DstY> <!-- unuse , reserved-->
<DstW>0</DstW> <!-- unuse , reserved-->
<DstH>0</DstH> <!-- unuse , reserved-->
</Connector>
</DisplayMode>
</HwComposerEnv>
```

4. 常见问题和需求处理

4.1 SERDES屏幕点不亮排查思路

1、首先确认Android是否正常显示

1) 使用PC工具可以预览到开机界面则表示正常

2) adb截图正常表示Android 显示正常;

2、确认serdes的I2C通信是否正常

```
cat /d/regmap/2-0040-max96755/registers
```

```
cat /d/regmap/2-0048-max96772/registers
```

```
cat /d/regmap/5-0042-max96745/registers
```

```
cat /d/regmap/5-0048-max96752/registers
```

如果正常，所有I2C结果都

3、显示summary是否生成

```
cat /d/dri/0/summary
```

如果生成但是对应的接口信息没有生成，则关注接口的频率是否能分出

```
cat /d/clk/clk_summary | grep vop
```

如果频率分不出来，对应的接口也不会初始化成功。

4、检查各DRM节点状态

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-DSI-1/status
```

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-DSI-2/status
```

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-eDP-1/status
```

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-eDP-2/status
```

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-DP-1/status
```

```
cat ./sys/devices/platform/display-subsystem/drm/card0/card0-DP-2/status
```

正常需要为connected状态，如果不是则排查serdes驱动接口是否返回正常。

注：其他问题处理方法后续版本再更新。